

API های مورد نیاز فرانت DeskRadar

Base URL پیشنهادی: `/api/v1`

قواعد عمومی:

- تاریخ ها ۸۶۰۱ ISO باشند؛ مثل `2026-09-01T10:30:00Z`.
- امتیازهای `confidence`، `score` و `similarity` بین `0` و `1` باشند.
- نقش، شناسه و نام کاربر از session گرفته شوند، نه body فرانت.
- پاسخ لیست های صفحه بندی شده:

```
{ items: T[]; page: number; pageSize: number; total: number }
```

احراز هویت

کاربرد	Endpoint	Method
ثبت نام کاربر عادی	<code>/auth/register</code>	POST
ورود	<code>/auth/login</code>	POST
بازیابی کاربر بعد از refresh	<code>/auth/me</code>	GET
خروج	<code>/auth/logout</code>	POST

ورودی ها:

```
// register
{ "name": "حسین احمدی", "email": "user@company.ir", "password": "secret123" }

// login
{ "email": "user@company.ir", "password": "secret123" }
```

پاسخ `register`، `login` و `me`:

```
{
  "user": {
    "id": "user-1",
    "name": "حسین احمدی",
    "email": "user@company.ir",
    "role": "user"
  }
}
```

پیشنهاد: session با cookie امن و `HttpOnly` نگه داری شود. ثبت نام همیشه نقش `user` می سازد. پاسخ logout می تواند `204` باشد.

دسترسی/کاربرد	Endpoint	Method
ادمین: همه تیکت‌ها؛ کاربر: فقط تیکت‌های خودش	<code>/tickets</code>	GET
ثبت تیکت توسط کاربر	<code>/tickets</code>	POST
جزئیات کامل تیکت، تحلیل و ارجاع	<code>/tickets/{id}</code>	GET
تغییر وضعیت توسط ادمین	<code>/tickets/{id}/status</code>	PATCH
تحلیل مجدد توسط ادمین	<code>/tickets/{id}/analyze</code>	POST
تأیید یا رد تحلیل توسط ادمین	<code>/tickets/{id}/analysis-feedback</code>	POST

نکات پاسخ

- `GET /tickets`: queryهای `q, category, urgency, status, page, pageSize` را بگیرد و `ListResponse<TicketSummary>` برگرداند. ترتیب پیش فرض جدیدترین اول باشد.
- `POST /tickets`: ورودی فقط `title, description, department` است. پاسخ 201 یک `TicketDetail` با `analysisStatus: "pending"`، `analysis: null` و `escalation: null` است. تحلیل باید خودکار شروع شود.
- `GET /tickets/{id}`: یک `TicketDetail` برگرداند. ارجاع و پیام‌هایش داخل فیلد `escalation` هستند؛ اگر ارجاعی نیست مقدار `null`. API جداگانه برای گرفتن ارجاع تیکت لازم نیست.
- `PATCH /tickets/{id}/status`: ورودی `{ "status": "in_progress" }` و پاسخ تیکت به‌روزشده باشد.
- `POST /tickets/{id}/analyze`: پاسخ 202 به شکل `{ "id": 101, "analysisStatus": "pending" }`.
- `analysis-feedback`: ورودی `{ "value": "up" }` با مقادیر `up | down`؛ پاسخ موفق 204.

ورودی ثبت تیکت:

```
{
  "title": "برقرار نمی‌شود VPN اتصال",
  "description": "از صبح خطای احراز هویت می‌بینم",
  "department": "واحد فروش"
}
```

وضعیت‌های تیکت:

open | in_progress | escalated | resolved | closed

ارجاع به کارشناس و گفتگو

دسترسی/کاربرد	Endpoint	Method
ارجاع تیکت توسط صاحب آن	<code>/tickets/{id}/escalations</code>	POST
اینباکس ارجاع‌های ادمین	<code>/escalations</code>	GET
جزئیات پرونده، تیکت و پیام‌ها	<code>/escalations/{id}</code>	GET
ارسال پیام توسط صاحب تیکت یا ادمین	<code>/escalations/{id}/messages</code>	POST
پذیرش یا بستن توسط ادمین	<code>/escalations/{id}/status</code>	PATCH

نکات پاسخ

- ساخت ارجاع ورودی `{ "reason": "..."` و خروجی `Escalation` دارد. هم‌زمان وضعیت تیکت `escalated` شود، `reason` اولین پیام باشد و برای ادمین هشدار ساخته شود.
- اگر همان تیکت ارجاع `waiting` یا `active` دارد، ارجاع جدید ساخته نشود و همان قبلی برگردد.
- `GET /escalations` queryهای `q, status, page, pageSize` و پاسخ `ListResponse<EscalationListItem>`. خلاصه تیکت و آخرین پیام داخل هر آیتم باشد تا درخواست اضافه لازم نشود.
- `GET /escalations/{id}` پاسخ `{ escalation: Escalation, ticket: TicketSummary }`.
- ارسال پیام ورودی `{ "text": "..."` و خروجی `Escalation` به‌روزشده دارد. اولین پاسخ ادمین، پرونده را `active` و ادمین را مسئول کند.
- تغییر وضعیت ورودی `{ "status": "active"` و خروجی `Escalation` به‌روزشده دارد. مقادیر مجاز ادمین `active | resolved` هستند.
- با `resolved` شدن ارجاع، تیکت مرتبط هم در همان transaction به `resolved` تغییر کند. پرونده بسته پیام جدید نپذیرد.

رخدادها

دسترسی/کاربرد	Endpoint	Method
لیست رخدادها برای ادمین	<code>/incidents</code>	GET
جزئیات رخداد و تیکت‌های مشابه	<code>/incidents/{id}</code>	GET
تأیید یا رفع رخداد توسط ادمین	<code>/incidents/{id}/status</code>	PATCH

- لیست queryهای `q, status, page, pageSize` و پاسخ `ListResponse<IncidentSummary>` دارد.
- جزئیات یک `Incident` کامل همراه تیکت‌های مرتبط و امتیاز `similarity` برمی‌گرداند.
- تغییر وضعیت ورودی `{ "status": "confirmed"` و خروجی رخداد به‌روزشده دارد. مقادیر: `candidate | resolved | confirmed`. هنگام رفع، سرور `resolvedAt` را ثبت کند.

دسترسی/کاربرد	Endpoint	Method
لیست و جستجو برای همه کاربران	<code>/knowledge-articles</code>	GET
جزئیات کامل مقاله	<code>/knowledge-articles/{id}</code>	GET
ساخت مقاله توسط ادمین	<code>/knowledge-articles</code>	POST

- لیست queryهای `q, category, page, pageSize` و پاسخ `ListResponse<KnowledgeArticleSummary>` دارد؛ `content` فقط در جزئیات لازم است.
- ساخت مقاله ورودی `title, summary, category, tags, content` و پاسخ `201` با مدل کامل مقاله دارد. `author` و `updatedAt` را سرور تعیین کند.

هشدارها

دسترسی/کاربرد	Endpoint	Method
هشدارهای ادمین؛ با <code>unread=true</code> فقط خوانده نشده ها	<code>/alerts</code>	GET
دریافت زنده هشدار جدید	<code>/ws/alerts</code>	WS

- به خاطر polling فعلی، `GET /alerts` مستقیماً `Alert[]` برگرداند، نه پاسخ صفحه بندی شده.
- هر پیام WebSocket دقیقاً یک `Alert` باشد. در صورت قطع اتصال، فرانت هر ۱۵ ثانیه `GET /alerts?unread=true` را صدا می زند.

داشبورد

کاربرد	Endpoint	Method
آمار خلاصه بر اساس نقش	<code>/dashboard/summary</code>	GET

`dashboard/summary` لازم است تا برای نمایش آمار، تمام صفحات لیست ها دانلود نشوند.

پاسخ ادمین:

```
{
  openTickets: number;
  urgentTickets: number;
  activeIncidents: number;
  waitingEscalations: number;
  categoryCounts: Record<TicketCategory, number>;
  recentAlerts: Alert[];
}
```

پاسخ کاربر:

```
{
  openTickets: number;
  resolvedTickets: number;
  activeEscalations: number;
}
```

```
recentTickets: TicketSummary[];  
}
```

مدل‌های مرجع

```
type Role = "admin" | "user";  
type TicketCategory = "vpn" | "email" | "network" | "printer" | "account";  
type Urgency = "low" | "medium" | "high" | "critical";  
type TicketStatus = "open" | "in_progress" | "escalated" | "resolved" | "closed";  
type AnalysisStatus = "pending" | "complete" | "failed";  
type EscalationStatus = "waiting" | "active" | "resolved";  
type IncidentStatus = "candidate" | "confirmed" | "resolved";
```

```
interface TicketSummary {  
  id: number;  
  title: string;  
  department: string;  
  requesterId: string;  
  requesterName: string;  
  category: TicketCategory;  
  categoryLabelFa: string;  
  urgency: Urgency;  
  status: TicketStatus;  
  analysisStatus: AnalysisStatus;  
  confidence: number;  
  createdAt: string;  
  updatedAt: string;  
}
```

```
interface TicketDetail extends TicketSummary {  
  description: string;  
  analysis: TicketAnalysis | null;  
  escalation: Escalation | null;  
}
```

```
interface TicketAnalysis {  
  category: TicketCategory;  
  categoryLabelFa: string;  
  intentLabelFa: string;  
  urgency: Urgency;  
  confidence: number;  
  summaryFa: string;  
  suggestedReplyFa: string;  
  reasonsFa: string[];  
  similarTickets: Array<{ id: number; title: string; score: number }>;  
  relatedArticle: { id: number; title: string; score: number } | null;  
  possibleIncident: boolean;  
}
```

```
interface EscalationMessage {  
  id: string;  
  senderId: string;  
  senderName: string;  
  senderRole: Role;  
  text: string;  
  createdAt: string;  
}
```

```
interface Escalation {  
  id: string;  
  ticketId: number;  
  requesterId: string;  
  requesterName: string;  
  reason: string;
```

```

status: EscalationStatus;
assignedAdminId?: string;
assignedAdminName?: string;
createdAt: string;
updatedAt: string;
messages: EscalationMessage[];
}

interface EscalationListItem extends Omit<Escalation, "messages"> {
  ticket: Pick<TicketSummary, "id" | "title" | "status" | "urgency">;
  lastMessage: EscalationMessage;
}

interface Incident {
  id: number;
  title: string;
  description: string;
  category: TicketCategory;
  categoryLabelFa: string;
  severity: "low" | "medium" | "high" | "critical";
  status: IncidentStatus;
  detectedReason: string;
  createdAt: string;
  resolvedAt?: string;
  tickets: Array<{ ticketId: number; title: string; similarity: number }>;
}

type IncidentSummary = Omit<Incident, "tickets"> & { ticketCount: number };

interface KnowledgeArticle {
  id: number;
  title: string;
  summary: string;
  category: TicketCategory;
  categoryLabelFa: string;
  tags: string[];
  content: string;
  updatedAt: string;
  author: string;
}

type KnowledgeArticleSummary = Omit<KnowledgeArticle, "content">;

interface Alert {
  id: string;
  title: string;
  message: string;
  severity: "low" | "medium" | "high" | "critical";
  type: "incident" | "sla" | "ticket" | "system" | "escalation";
  createdAt: string;
  read: boolean;
  href?: string;
}

```

قوانین دسترسی

- کاربر فقط تیکت‌ها و ارجاع‌های خودش را ببیند.
- فقط صاحب تیکت بتواند آن را ارجاع دهد یا در گفتگوی آن پیام بفرستد.
- فقط ادمین وضعیت تیکت، ارجاع و رخداد را تغییر دهد.
- فقط ادمین مقاله بسازد و هشدارها را ببیند.
- امنیت در بک‌اند enforce شود و به مخفی بودن صفحات فرانت وابسته نباشد.